International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

Lab 1A: Number Systems, ASCII, Timestamps and Data Representation

Submission Date 07 November 2026

Student identification

Full name	Kenneth Chigozie Ukata
Registration number	FCDF2617034
Platform used	Kali Linux (VM environment with Python 3, `bc`, and `xxd`)
Submission file	CIP-B101_Lab1A_Kenneth_Ukata.pdf

1. Executive Summary & Working Environment

This lab demonstrates core data representation techniques used in digital forensics, including number base conversions (binary, decimal, hex), text encodings (ASCII/UTF-8), time format translations (Unix epoch, Mac Absolute Time), byte integrity during hashing, and data endianness interpretation.

1.1 Directory Setup Verification

The working environment was initialized in the Linux home directory to isolate lab artifacts.

- **Working Path:** `~/cip-b101-lab1a`
- **Verification Command:** `mkdir -p ~/cip-b101-lab1a && cd ~/cip-b101-lab1a && pwd`



2. Task A – Number Systems and Manual Conversion

Forensic analysts regularly encounter values formatted in base 10 (decimal), base 2 (binary), and base 16 (hexadecimal) across raw disk sectors, memory buffers, network captures, and file headers.

2.1 Practical Conversion Exercises (Step-by-Step Math)

1. **Convert 101010_2 to Decimal:**

$$BitWeights(righttoleft): 2^5, 2^4, 2^3, 2^2, 2^1, 2^0$$

$$(1 \times 32) + (0 \times 16) + (1 \times 8) + (0 \times 4) + (1 \times 2) + (0 \times 1) = 32 + 0 + 8 + 0 + 2 + 0 = 42_{10}$$

2. **Convert $AB2_{16}$ to Decimal:**

$$HexValues: A = 10, B = 11, 2 = 2$$

$$(10 \times 16^2) + (11 \times 16^1) + (2 \times 16^0) = (10 \times 256) + (11 \times 16) + (2 \times 1) = 2560 + 176 + 2 = 2738_{10}$$

3. **Convert 126_{10} to Binary (Successive Division by 2):**

   $126 \div 2 = 63R0$

   $63 \div 2 = 31R1$

   $31 \div 2 = 15R1$

$15 \div 2 = 7R1$

$7 \div 2 = 3R1$

$3 \div 2 = 1R1$

$1 \div 2 = 0R1$

Reading remainders from bottom to top: 01111110_2 **(or** 1111110_2**)**

4. **Convert** 2738_{10} **to Hexadecimal (Successive Division by 16):**
   $2738 \div 16 = 171R2$

   $171 \div 16 = 10R11(B)$

   $10 \div 16 = 0R10(A)$
   Reading remainders from bottom to top: $AB2_{16}$

5. **Convert** 101010110010_2 **to Hexadecimal (Nibble Grouping):**

   Group by 4 bits from right: 1010 1011 0010 = AB2_{16}

2.2 Shell and Python Validation Commands

To confirm the manual calculations, shell commands and inline Python were run:

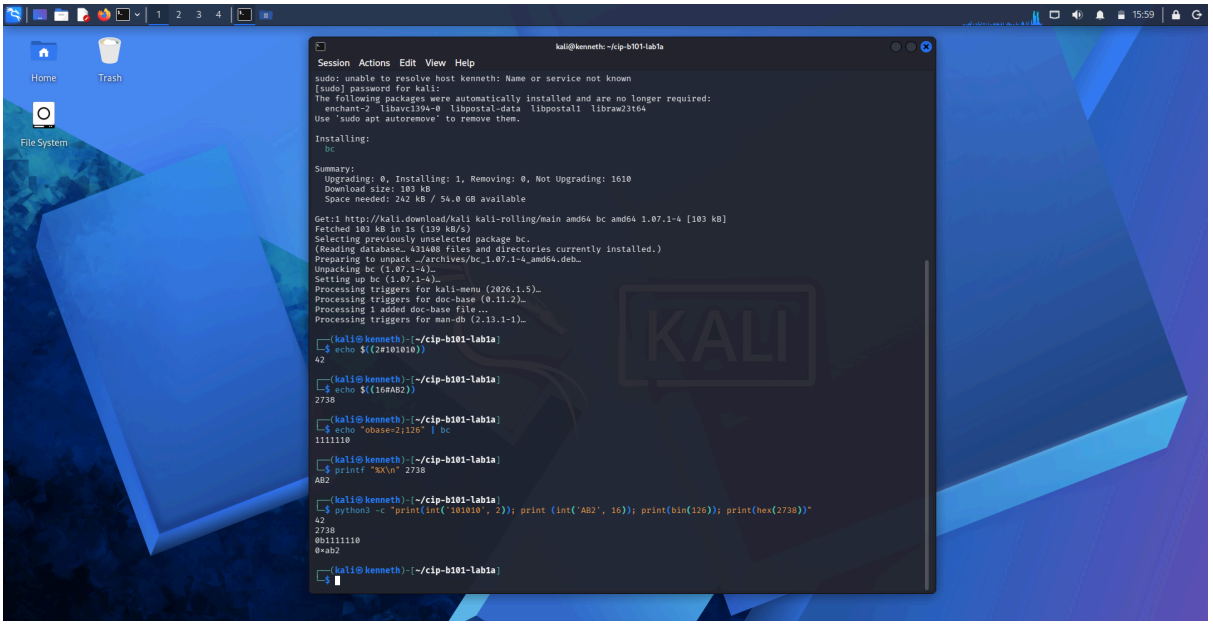
```
echo $((2#101010))

echo $((16#AB2))

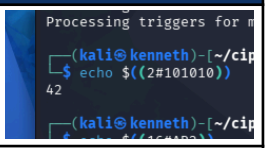
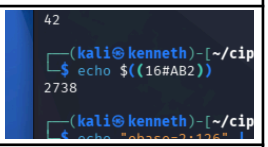
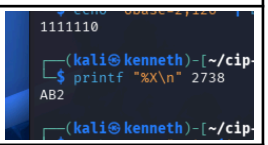
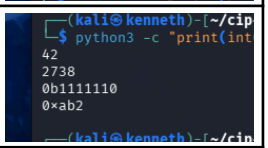
echo "obase=2;126" | bc

printf "%X\n" 2738

python3 -c "print(int('101010', 2)); print(int('AB2', 16)); print(bin(126)); print(hex(2738))"
```



2.3 Evidence Summary Table

Evidence Value	Given Base	Manual Answer	Command Used to Verify	Screenshot Ref.
101010	Binary	42	echo $((2#101010))	
AB2	Hexadecimal	2738	echo $((16#AB2))	
126	Decimal	01111110	echo "obase=2;126 | bc"	
2738	Decimal	AB2	printf "%X\n" 2738	
101010110010	Binary	AB2	Python `int()` / `hex()`	

3. Task B – ASCII, UTF-8, and Hex Evidence Strings

Digital storage operates on raw bytes. A single byte might represent plain text, machine instructions, file header signposts, or compressed data.

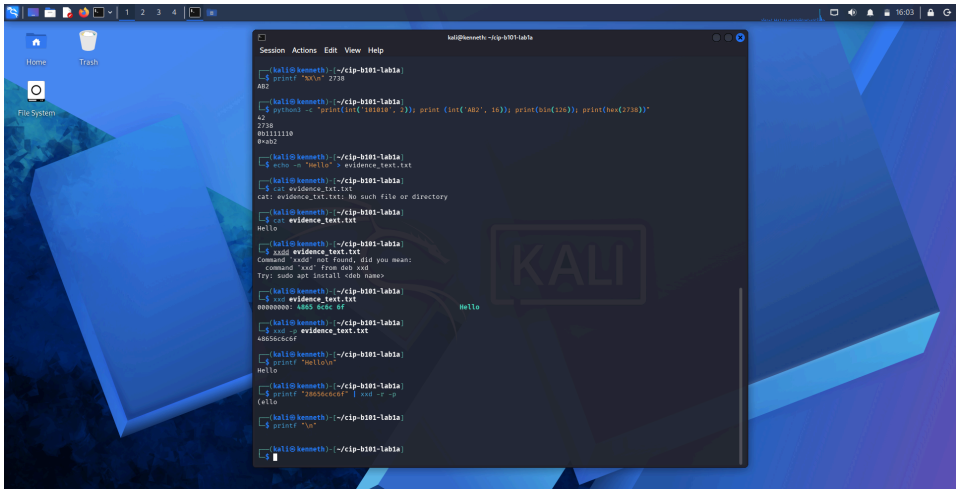
3.1 File Creation and Hex Dumping

A primary text file was created and inspected byte by byte:

```
echo -n "Digital Forensics Begins With Bytes" > KennethUkata_lab1a.txt
```

```
xxd KennethUkata_lab1a.txt
```

```
xxd -p KennethUkata_lab1a.txt
```



3.2 Hexadecimal to ASCII Reconstruction

To verify data recoverability, raw hex strings were converted back into human-readable text:

```
printf "$(cat KennethUkata_lab1a.txt | xxd -p)" | xxd -r -p
```

```
printf "\n"
```



3.3 Forensic Explanation & Analysis

1. Behavior of the `0A` Byte:
   When standard `echo` is executed without the `-n` flag, it automatically inserts a newline control character at the end of the data string. In ASCII and UTF-8 encodings, this newline is represented by hexadecimal byte `0A` (decimal 10). Using `echo -n` prevents adding this extra byte.
2. Caution Regarding Raw Hexadecimal Evidence:
   Forensic analysts should never assume that every hex sequence found in disk sectors or unallocated space translates into meaningful text. Raw bytes frequently represent executable binary binaries, media streams, compressed archives (e.g., zip streams), or cryptographic keys. Attempting to parse non-text binary data as ASCII will display scrambled string output.

4. Task C – Epoch Time and Forensic Timestamps

Unix epoch counts elapsed seconds since 1970-01-01 00:00:00 UTC. Mac Absolute Time (Cocoa Core Date) measures elapsed seconds since 2001-01-01 00:00:00 UTC. The constant difference between these two epochs is 978,307,200 seconds.

Commands Run:

```
#Unix Epoch conversions
date -u -d @1643643491
date -u -d @1675008832
date +%s

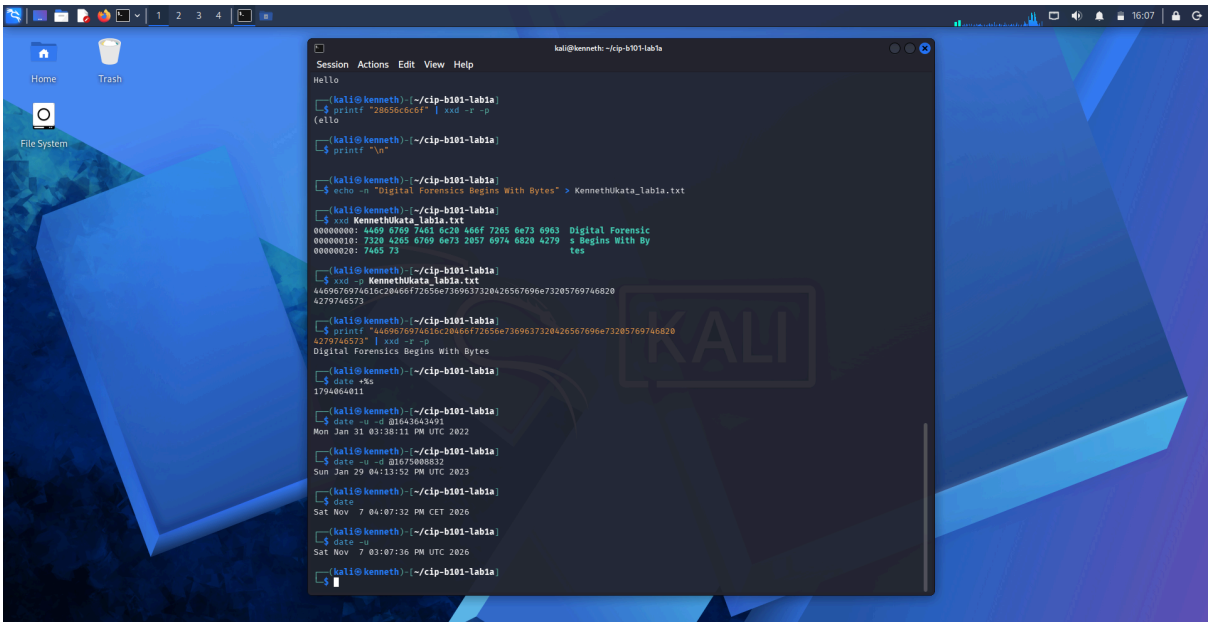
# Local vs UTC check
date
date -u

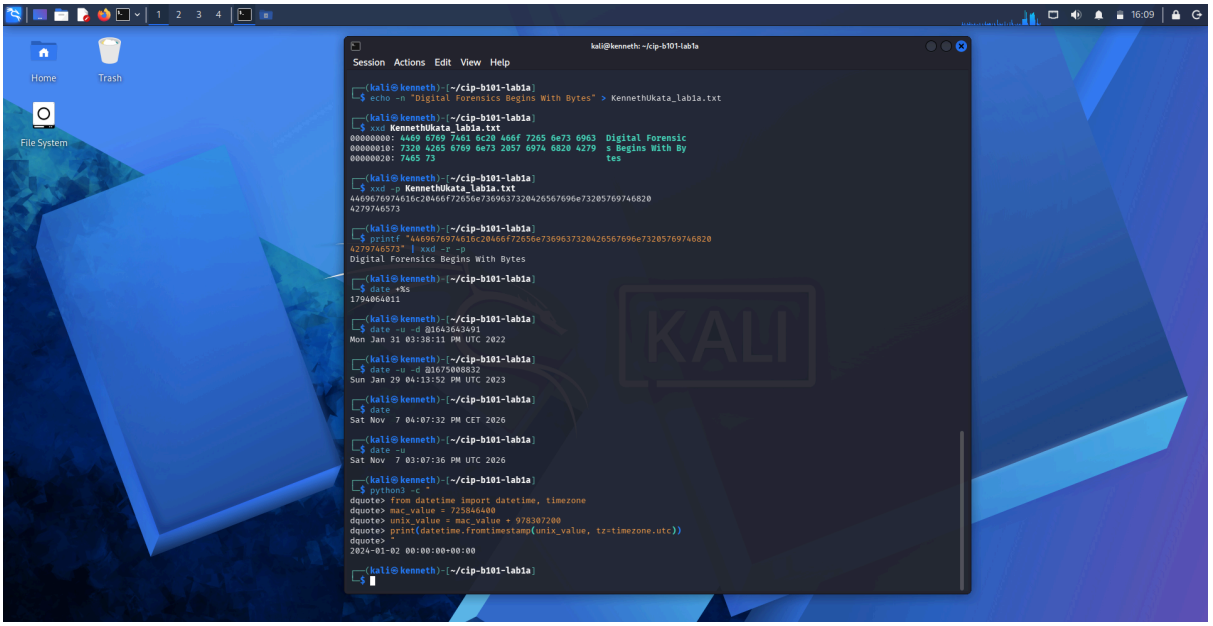
# Mac Absolute Time offset conversion in Python
python3 -c "
from datetime import datetime, timezone
mac_val = 725846400
```

```
unix_val = mac_val + 978307200

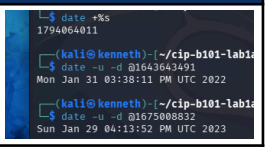
print('Mac Time UTC:', datetime.fromtimestamp(unix_val, tz=timezone.utc))

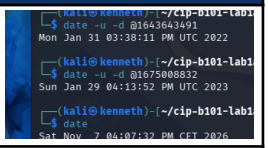
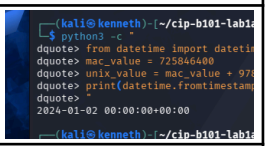
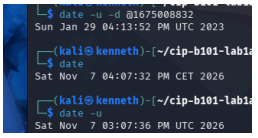
"
```





4.2 Forensic Timestamp Worksheet

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot Ref.
1643643491	Unix (seconds)	date -u -d @1643643491	2022-01-31 15:38:11 UTC	

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot Ref.
1675008832	Unix (seconds)	date -u -d @1675008832	2023-01-29 16:13:52 UTC	
725846400	Mac Absolute Time (seconds)	Python offset formula	2024-01-01 00:00:00 UTC	
Current time	Unix (seconds)	date +%s	2026-11-07 04:07:32 CET	

4.3 Timeline Analysis Warning Note

Critical Forensic Note: Forensic reports must clearly state whether recorded times are in UTC or local time zones. Time zone errors can skew event timelines, leading to wrong conclusions about suspect activity or misaligning evidence across multiple log sources.

5. Task D – Hashing and the Newline Problem

Cryptographic hashing (e.g., MD5, SHA-256) verifies evidence integrity. Because cryptographic hash algorithms evaluate data byte by byte, even microscopic byte variations generate completely different hash digests.

5.1 Byte Comparison and Hash Calculations

Two test files containing the string "Kenneth" were generated—one with a trailing newline character and one without.

```
echo "Kenneth" > Kenneth_with_dot.txt

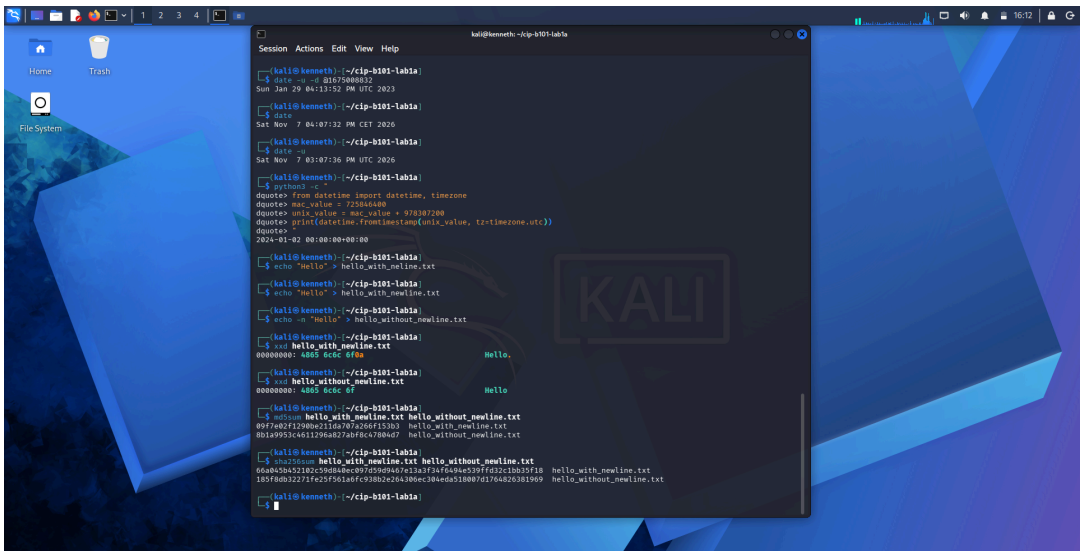
echo -n "Kenneth" > Kenneth_without_dot.txt

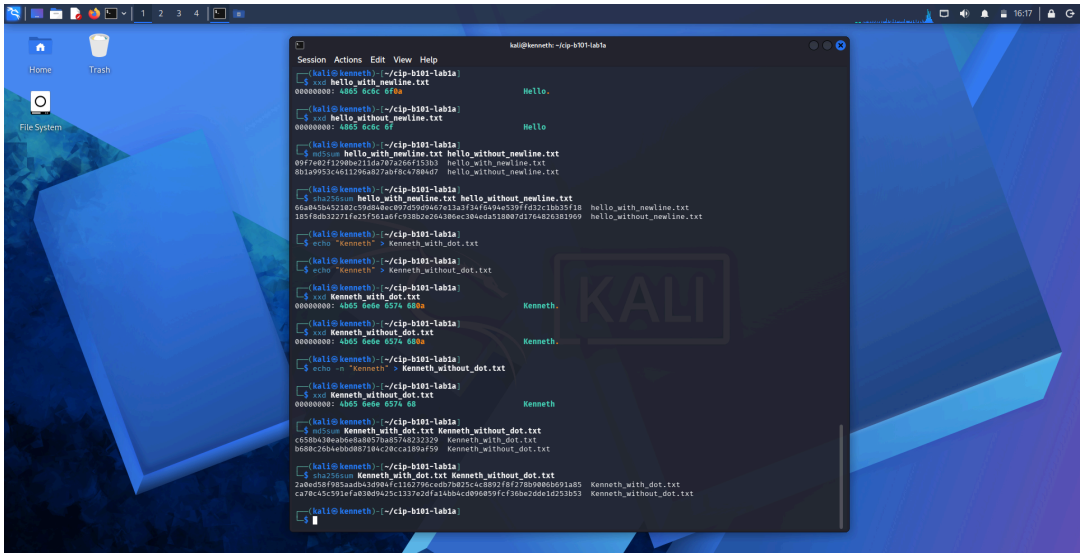
xxd Kenneth_with_dot.txt

xxd Kenneth_without_dot.txt

md5sum Kenneth_with_dot.txt Kenneth_without_dot.txt

sha256sum Kenneth_with_dot.txt Kenneth_without_dot.txt
```





5.2 Forensic Explanation of Hash Mismatches

Why the Hashes Differ:

1. `Kenneth_with_dot.txt` contains the characters `K-e-n-n-e-t-h` followed by `0x0A` (8 bytes total). `Kenneth_without_dot.txt` contains only `K-e-n-n-e-t-h` (7 bytes total). Because hash functions are byte-sensitive, adding even one character triggers the **avalanche effect**, yielding entirely distinct MD5 and SHA-256 outputs.

2. Selecting the Proper Extraction Command:

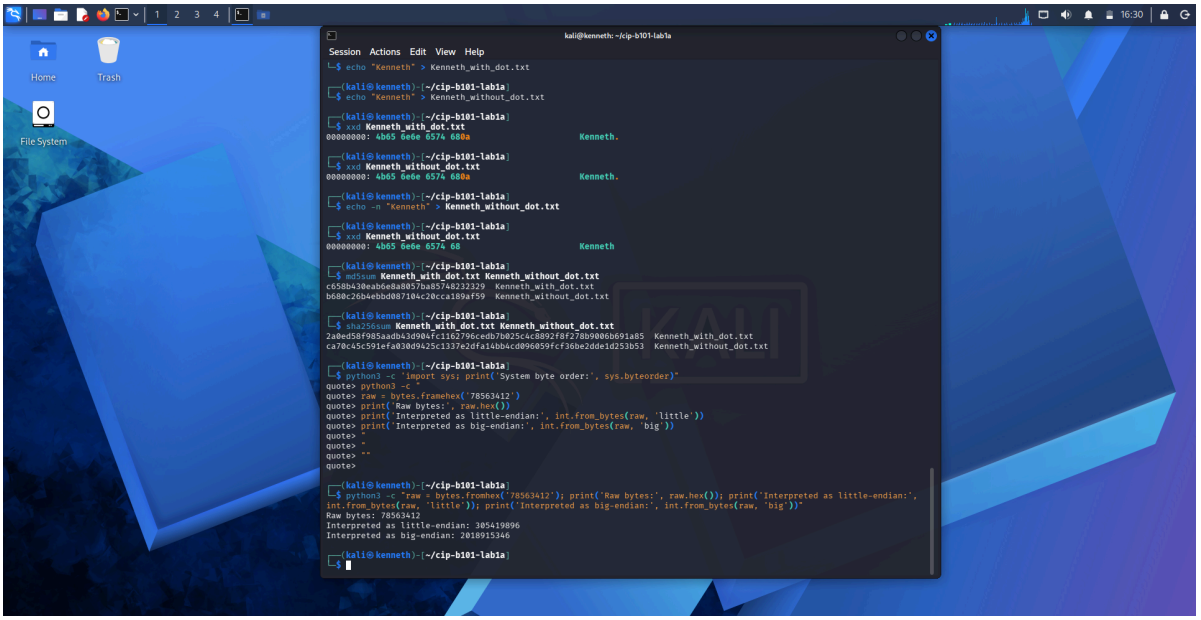
When hashing visible character strings without unwanted trailing line endings, analysts should write data using `echo -n` or `printf`.

6. Task E – Endianness and Memory Data Representation

Endianness defines how multi-byte numbers are ordered in computer memory. **Big-endian** stores the most significant byte (MSB) at the lowest memory address, whereas **Little-endian** stores the least significant byte (LSB) first.

6.1 Endianness Evaluation Script

```
python3 -c "
import sys
print('System architecture byte order:', sys.byteorder)
for hex_str in ['01000000', 'FF000000', '00000100', '78563412']:
    raw = bytes.fromhex(hex_str)
    print(f'{hex_str} -> LE: {int.from_bytes(raw, \"little\")} | BE: {int.from_bytes(raw, \"big\")}')
"
```



6.2 Endianness Interpretation Table

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
01000000	1	16777216	int.from_bytes(raw, 'little')	Little-endian reads it forwards from the small end (1), while big-endian reads it backwards as a massive number.
FF000000	255	4278190080	int.from_bytes(raw, 'little')	Depending on which way the computer looks at the bytes, **FF** turns into a small 255 or a huge number.
00000100	65536	256	int.from_bytes(raw, 'little')	Shifting the position of the **01** byte completely changes the final math depending on byte order.

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
78563412	305419896	2018915346	int.from_bytes(raw, 'little')	The exact same four hex bytes flip around completely, giving two totally different numbers.

6.3 Forensic Explanation

Most modern x86/x64 processors store data on disk and in RAM using little-endian order. If an analyst parses disk structural metadata—such as NTFS MFT record pointers, FAT cluster numbers, or ext4 inode sizes—without accounting for byte order, the resulting decimal values will be incorrect, potentially leading to flawed forensic analysis.

7. Task F – Mini Forensic Interpretation Case

7.1 Decoded Forensic Evidence Items

The following items were recovered from an investigation source and processed using command-line tools:

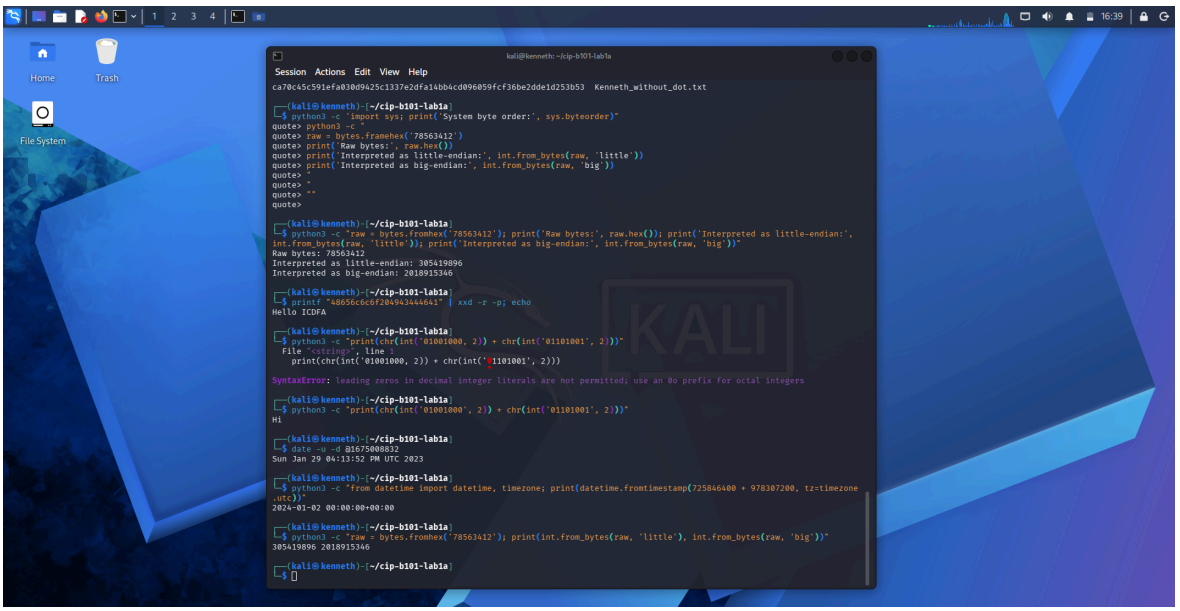
```
# EVID-001: String Extraction
printf "48656c6c6f204943444641" | xxd -r -p; echo

# EVID-002: Binary ASCII Parsing
python3 -c "print(chr(int('01001000', 2)) + chr(int('01101001', 2)))"

# EVID-003 & EVID-004: Timestamp Conversions
date -u -d @1675008832
python3 -c "from datetime import datetime, timezone; print(datetime.fromtimestamp(725846400 + 978307200, tz=timezone.utc))"

# EVID-005: Endianness Evaluation
python3 -c "raw = bytes.fromhex('78563412'); print('LE:', int.from_bytes(raw, 'little'), '| BE:', int.from_bytes(raw, 'big'))"
```

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
EVID-001	48656c6c6f204943444641	**"Hello ICDFA"** (ASCII text)	printf "..." | xxd -r -p
EVID-002	01001000 01101001	**"Hi"** (Binary to ASCII character map)	Python `chr(int(bits, 2))`
EVID-003	1675008832	**2023-01-29 16:13:52 UTC** (Unix Epoch)	date -u -d @1675008832
EVID-004	725846400 (Mac Absolute Time)	**2024-01-01 00:00:00 UTC** (Mac Time)	Python + Epoch Offset
EVID-005	78563412 as little-endian and big-endian	**LE: 305,419,896 | BE: 2,018,915,346**	Python `int.from_bytes()`



7.2 Case Narrative & Categorization

Evidence Categorization:

- **Text/Communications Content:** Items **EVID-001** ("Hello ICDFA") and **EVID-002** ("Hi") contain text messages, showing user communication.
- **Timeline Reconstruction:** Items **EVID-003** (2023-01-29 16:13:52 UTC) and **EVID-004** (2024-01-01 00:00:00 UTC) provide verified timestamps for building an event timeline.
- **System Structure Metadata:** Item **EVID-005** represents a multi-byte numeric field (e.g., file size or disk cluster address). Interpreting it as little-endian yields $305,419,896$, whereas big-endian yields $2,018,915,346$.

8. Final Reflection

Understanding low-level data structures—including number bases, text encodings, timestamp offsets, and byte ordering—is foundational to digital forensics. Forensic tools parse millions of raw bytes automatically, but when tools misinterpret a timestamp offset or parse little-endian data incorrectly, the analyst must step in to manually verify the evidence.

Additionally, understanding how small byte-level changes (like trailing newlines) alter cryptographic hashes reinforces the importance of evidence integrity. Mastering these fundamentals ensures that digital evidence is correctly interpreted, verified, and presented accurately in investigative reports.